

Automaton

Генериран от Doxygen 1.17.0

1 Класове Йерархичен указател	1
1.1 Класове Йерархия	1
2 Класове Указател	3
2.1 Класове Списък	3
3 Файлове Списък	5
3.1 Файлове Списък	5
4 Класове Документация	7
4.1 Automaton Клас Препратка	7
4.1.1 Подробно описание	8
4.1.2 Конструктор & Деструктор Документация	8
4.1.2.1 Automaton()	8
4.1.3 Членове Функции(методи) Документация	9
4.1.3.1 addTransition()	9
4.1.3.2 createState()	9
4.1.3.3 getAlphabet()	9
4.1.3.4 getReversed()	10
4.1.3.5 getStatesCount()	10
4.1.3.6 isDeterministic()	10
4.1.3.7 recognise()	10
4.1.4 Приятели и Свързани функции Документация	11
4.1.4.1 Concat	11
4.1.4.2 Star	11
4.1.4.3 Union	11
4.2 AutomatonCLI Клас Препратка	12
4.2.1 Подробно описание	12
4.3 AutomatonSerializer Клас Препратка	13
4.3.1 Членове Функции(методи) Документация	13
4.3.1.1 serialize()	13
4.4 BadSymbolException Клас Препратка	13
4.4.1 Подробно описание	14
4.5 BinaryOperationRegEx Клас Препратка	14
4.5.1 Подробно описание	15
4.6 ConcatRegEx Клас Препратка	15
4.6.1 Подробно описание	16
4.6.2 Членове Функции(методи) Документация	16
4.6.2.1 clone()	16
4.6.2.2 eval()	17
4.6.2.3 toAutomaton()	17
4.6.2.4 toString()	17
4.7 Exception Клас Препратка	17
4.7.1 Подробно описание	18

4.8 LiteralRegEx Клас Препратка	18
4.8.1 Подробно описание	18
4.8.2 Членове Функции(методи) Документация	19
4.8.2.1 clone()	19
4.8.2.2 eval()	19
4.8.2.3 toAutomaton()	19
4.8.2.4 toString()	19
4.9 ParserException Клас Препратка	19
4.9.1 Подробно описание	20
4.9.2 Членове Функции(методи) Документация	20
4.9.2.1 toString()	20
4.10 RangeRegEx Клас Препратка	20
4.10.1 Подробно описание	21
4.10.2 Членове Функции(методи) Документация	21
4.10.2.1 clone()	21
4.10.2.2 eval()	21
4.10.2.3 toAutomaton()	21
4.10.2.4 toString()	22
4.11 RegEx Клас Препратка	22
4.11.1 Подробно описание	22
4.11.2 Членове Функции(методи) Документация	23
4.11.2.1 eval()	23
4.11.2.2 toAutomaton()	23
4.11.2.3 toString()	23
4.12 RegExParser Клас Препратка	23
4.12.1 Подробно описание	24
4.12.2 Членове Функции(методи) Документация	24
4.12.2.1 parse()	24
4.13 StarRegEx Клас Препратка	24
4.13.1 Подробно описание	25
4.13.2 Членове Функции(методи) Документация	25
4.13.2.1 clone()	25
4.13.2.2 eval()	25
4.13.2.3 toAutomaton()	26
4.13.2.4 toString()	26
4.14 StateExistsException Клас Препратка	26
4.14.1 Подробно описание	27
4.14.2 Членове Функции(методи) Документация	27
4.14.2.1 toString()	27
4.15 Transition Структура Препратка	27
4.15.1 Подробно описание	27
4.16 UnionRegEx Клас Препратка	28
4.16.1 Подробно описание	29

4.16.2 Членове Функции(методи) Документация	29
4.16.2.1 clone()	29
4.16.2.2 eval()	29
4.16.2.3 toAutomaton()	29
4.16.2.4 toString()	29
5 Файлове Документация	31
5.1 Automaton.hpp	31
5.2 AutomatonSerializer.hpp	32
5.3 BadSymbolException.hpp	33
5.4 Exception.hpp	33
5.5 ParserException.hpp	33
5.6 StateExistsException.hpp	33
5.7 BinaryOperationRegEx.hpp	34
5.8 ConcatRegEx.hpp	34
5.9 LiteralRegEx.hpp	34
5.10 RangeRegEx.hpp	34
5.11 RegEx.hpp	35
5.12 RegExParser.hpp	35
5.13 StarRegEx.hpp	35
5.14 UnionRegEx.hpp	36
5.15 AutomatonCLI.hpp	36
Азбучен указател	39

Глава 1

Класове Йерархичен указател

1.1 Класове Йерархия

Този списък с наследявания е сортиран, но не изцяло по азбучен ред:

Automaton	7
AutomatonCLI	12
AutomatonSerializer	13
Exception	17
BadSymbolException	13
ParserException	19
StateExistsException	26
Regex	22
BinaryOperationRegex	14
ConcatRegex	15
UnionRegex	28
LiteralRegex	18
RangeRegex	20
StarRegex	24
RegexParser	23
Transition	27

Глава 2

Класове Указател

2.1 Класове Списък

Класове, структури, обединения и интерфейси с кратко описание:

Automaton

Класът [Automaton](#) представлява краен автомат, който може да бъде недетерминиран (NFA) или детерминиран (DFA). Той съдържа състоянията и преходите между тях. Класът предоставя методи за създаване на нови състояния, добавяне на преходи, разпознаване на думи, обръщане на автомата, детерминизация и минимизация. Също така включва функции за обединение, конкатенация и звезда на автомати

7

AutomatonCLI

Класът [AutomatonCLI](#) предоставя интерфейс за команден ред, който позволява на потребителите да взаимодействат с автомати и регулярни изрази. Чрез този интерфейс потребителите могат да създават, модифицират, запазват и зареждат автомати, както и да извършват различни операции като детерминизация, минимизация, обединение, конкатенация и звезда. CLI-то също така поддържа разпознаване на думи от автоматите и визуализация на автоматите чрез DOT формат

12

AutomatonSerializer

13

BadSymbolException

Изключение, което се хвърля при опит за използване на символ, който не е част от азбуката на автомата. Съдържа информация за символа, който е бил използван и не е валиден

13

BinaryOperationRegex

Класът [BinaryOperationRegex](#) е абстрактен базов клас за представяне на бинарни операции между регулярни изрази, като обединение и конкатенация. Той съдържа два указателя към обекти от тип [Regex](#), които представляват лявата и дясната част на операцията

14

ConcatRegex

Класът [ConcatRegex](#) е конкатенация на два други регулярни изрази

15

Exception

Базов клас за всички изключения в проекта. Той съдържа съобщение за грешката и предоставя метод `toString()` за връщане на това съобщение

17

LiteralRegex

Класът [LiteralRegex](#) представлява регулярни изрази, които съответстват на конкретна дума

18

ParserException

Изключение, което се хвърля при срещане на синтактична грешка по време на парсане на регулярни изрази. Съдържа текст описващ грешката, индекса където е възникнала грешката, и самия израз, който е бил парсван

19

RangeRegex	Класът RangeRegex представлява регулярни изрази, които са диапазон от символи. Например, [a-z] представлява всички малки букви от а до z	20
Regex	Абстрактен базов клас за представяне на регулярни изрази	22
RegexParser	Класът RegexParser използва рекурсивен парсер за обработка на различните оператори в регулярните изрази, като обединение (+), конкатенация(без оператор) и звезда (*). За празна дума се използва символът '@'	23
StarRegex	Класът StarRegex представлява регулярни изрази, които са звезда на друг регулярен израз	24
StateExistsException	Изключение, което се хвърля при опит за създаване на състояние с идентификатор, който вече съществува в автомата, или при опит за достъп до несъществуващо състояние. Съдържа информация за идентификатора на състоянието и дали то вече съществува или не	26
Transition	Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като '\0')	27
UnionRegex	Класът UnionRegex представлява обединение на два други регулярни изрази . .	28

Глава 3

Файлове Списък

3.1 Файлове Списък

Пълен списък с документираните файлове с кратко описание:

Automaton.hpp	31
AutomatonSerializer.hpp	32
BadSymbolException.hpp	33
Exception.hpp	33
ParserException.hpp	33
StateExistsException.hpp	33
BinaryOperationRegEx.hpp	34
ConcatRegEx.hpp	34
LiteralRegEx.hpp	34
RangeRegEx.hpp	34
RegEx.hpp	35
RegExParser.hpp	35
StarRegEx.hpp	35
UnionRegEx.hpp	36
AutomatonCLI.hpp	36

Глава 4

Класове Документация

4.1 Automaton Клас Препратка

Класът [Automaton](#) представлява краен автомат, който може да бъде недетерминиран (NFA) или детерминиран (DFA). Той съдържа състоянията и преходите между тях. Класът предоставя методи за създаване на нови състояния, добавяне на преходи, разпознаване на думи, обръщане на автомата, детерминизация и минимизация. Също така включва функции за обединение, конкатенация и звезда на автомати.

```
#include <Automaton.hpp>
```

Общодостъпни членове функции

- [Automaton](#) (const std::vector< char > alphabet)
Конструктор, който приема азбука от символи и създава празен автомат с тази азбука. Автоматът няма състояния или преходи при инициализация.
- void [createNewState](#) (size_t id, bool isStart, bool isFinal)
Създава ново състояние с даден идентификатор и го добавя към автомата. Може да се укаже дали състоянието е начално и/или крайно.
- void [addTransition](#) (size_t from, char symbol, size_t to)
Добавя преход от едно състояние към друго с даден символ. Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като ^0).
- bool [recognise](#) (const std::string &word) const
Проверява дали дадена дума се разпознава от автомата.
- [Automaton getReversed](#) () const
Връща нов автомат, който е обърнатата версия на текущия автомат. В обърнатия автомат всички преходи са обърнати, началните състояния стават крайни и крайни състояния стават начални.
- void [reverse](#) ()
Обръща текущия автомат. Всички преходи се обръщат, началните състояния стават крайни и крайни състояния стават начални.
- bool [isDeterministic](#) () const
Проверява дали автоматът е детерминиран. Автоматът е детерминиран, ако няма две прехода от едно и също състояние с един и същ символ и няма епсилон-преходи.
- [Automaton getDeterministic](#) () const
Връща нов автомат, който е детерминизираната версия на текущия автомат. Детерминизацията се извършва чрез алгоритъма за конвертиране на NFA в DFA.
- void [determinise](#) ()

- Детерминира текущия автомат.
- `Automaton getMinimised () const`
Връща нов автомат, който е минимизираната версия на текущия автомат. Минимизацията се извършва чрез алгоритъма на Бжозовски
- `void minimise ()`
Минимизира текущия автомат. Минимизацията се извършва чрез алгоритъма на Бжозовски
- `const std::vector< char > &getAlphabet () const`
- `size_t getStatesCount () const`

Приатели

- `class AutomatonSerializer`
- `Automaton Union (const Automaton &lhs, const Automaton &rhs)`
Функция за обединение на два автомата.
- `Automaton Concat (const Automaton &lhs, const Automaton &rhs)`
Функция за конкатенация на два автомата.
- `Automaton Star (const Automaton &automaton)`
Функция за звезда на Клини на автомат.

4.1.1 Подробно описание

Класът `Automaton` представлява краен автомат, който може да бъде недетерминиран (NFA) или детерминиран (DFA). Той съдържа състоянията и преходите между тях. Класът предоставя методи за създаване на нови състояния, добавяне на преходи, разпознаване на думи, обръщане на автомата, детерминизация и минимизация. Също така включва функции за обединение, конкатенация и звезда на автомати.

4.1.2 Конструктор & Деструктор Документация

4.1.2.1 Automaton()

```
Automaton::Automaton (
    const std::vector< char > alphabet)
```

Конструктор, който приема азбука от символи и създава празен автомат с тази азбука. Автоматът няма състояния или преходи при инициализация.

Аргументи

alphabet	Вектор от символи, представляващи азбуката на автомата.
----------	---

4.1.3 Членове Функции(методи) Документация

4.1.3.1 addTransition()

```
void Automaton::addTransition (
    size_t from,
    char symbol,
    size_t to)
```

Добавя преход от едно състояние към друго с даден символ. Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като '\0').

Аргументи

from	Идентификатор на състоянието, от което започва преходът.
symbol	Символ, който се използва за прехода. Той трябва да е част от азбуката на автомата или да бъде епсилон.
to	Идентификатор на състоянието, към което води преходът.

4.1.3.2 createState()

```
void Automaton::createNewState (
    size_t id,
    bool isStart,
    bool isFinal)
```

Създава ново състояние с даден идентификатор и го добавя към автомата. Може да се укаже дали състоянието е начално и/или крайно.

Аргументи

id	Идентификатор на новото състояние. Той трябва да е уникален в рамките на автомата.
isStart	Булева стойност, указваща дали новото състояние е начално.
isFinal	Булева стойност, указваща дали новото състояние е крайно.

Изключения

StateExistsException	Ако състоянието с дадения идентификатор вече съществува в автомата, се хвърля изключение от тип StateExistsException .
BadSymbolException	Ако символът, използван в прехода, не е част от азбуката на автомата и не е епсилон, се хвърля изключение от тип BadSymbolException .

4.1.3.3 getAlphabet()

```
const std::vector< char > & Automaton::getAlphabet () const [inline]
```

Връща

азбуката на автомата като вектор от символи.

4.1.3.4 getReversed()

`Automaton` `Automaton::getReversed () const`

Връща нов автомат, който е обърнатата версия на текущия автомат. В обърнатия автомат всички преходи са обърнати, началните състояния стават крайни и крайни състояния стават начални.

Връща

Нов автомат, който е обърнатата версия на текущия автомат.

4.1.3.5 getStatesCount()

`size_t` `Automaton::getStatesCount () const` `[inline]`

Връща

Броя на състоянията в автомата.

4.1.3.6 isDeterministic()

`bool` `Automaton::isDeterministic () const`

Проверява дали автоматът е детерминиран. Автоматът е детерминиран, ако няма две прехода от едно и също състояние с един и същ символ и няма епсилон-преходи.

Връща

`true` - ако автоматът е детерминиран.

4.1.3.7 recognise()

`bool` `Automaton::recognise (`
 `const std::string & word) const`

Проверява дали дадена дума се разпознава от автомата.

Аргументи

<code>word</code>	Низ, представляващ думата, която трябва да се провери.
-------------------	--

4.1.4 Приятели и Свързани функции Документация

4.1.4.1 Concat

```
Automaton Concat (  
    const Automaton & lhs,  
    const Automaton & rhs) [friend]
```

Функция за конкатенация на два автомата.

Връща

Нов автомат(Недетерминиран), който представлява конкатенацията на двата входни автомата. Конкатенацията запазва всички състояния и преходи от двата автомата и добавя епсилон-преходи от всички крайни състояния на първия автомат към началните състояния на втория автомат. Новите начални са началните на първия автомат, а новите финални - финалните на втория автомат.

4.1.4.2 Star

```
Automaton Star (  
    const Automaton & automaton) [friend]
```

Функция за звезда на Клини на автомат.

Връща

Нов автомат(Недетерминиран), който представлява звездата на входния автомат. Звездата запазва всички състояния и преходи от входния автомат и добавя ново начално състояние, което е също така крайно. Добавя епсилон-преходи от това ново начално състояние към всички начални състояния на входния автомат, както и епсилон-преходи от всички крайни състояния на входния автомат към това ново начално състояние.

4.1.4.3 Union

```
Automaton Union (  
    const Automaton & lhs,  
    const Automaton & rhs) [friend]
```

Функция за обединение на два автомата.

Връща

Нов автомат(Недетерминиран), който представлява обединението на двата входни автомата. Обединението запазва всички състояния и преходи от двата автомата като новите начални са началните на двата автомата, а новите финални - финалните на двата автомата.

Документация за клас генериран от следните файлове:

- Automaton.hpp
- Automaton.cpp

4.2 AutomatonCLI Клас Препратка

Класът [AutomatonCLI](#) предоставя интерфейс за команден ред, който позволява на потребителите да взаимодействат с автомати и регулярни изрази. Чрез този интерфейс потребителите могат да създават, модифицират, запазват и зареждат автомати, както и да извършват различни операции като детерминизация, минимизация, обединение, конкатенация и звезда. CLI-то също така поддържа разпознаване на думи от автоматите и визуализация на автоматите чрез DOT формат.

```
#include <AutomatonCLI.hpp>
```

Общодостъпни членове функции

- void run ()
- void helpCmd ()
- void exitCmd ()
- void openCmd ()
- void saveCmd ()
- void saveAllCmd ()
- void listCmd ()
- void printCmd ()
- void removeCmd ()
- void resetCmd ()
- void deterministicCmd ()
- void determiniseCmd ()
- void reverseCmd ()
- void minimiseCmd ()
- void recogniseCmd ()
- void unionCmd ()
- void concatCmd ()
- void starCmd ()
- void regCmd ()
- void drawCmd ()

4.2.1 Подробно описание

Класът [AutomatonCLI](#) предоставя интерфейс за команден ред, който позволява на потребителите да взаимодействат с автомати и регулярни изрази. Чрез този интерфейс потребителите могат да създават, модифицират, запазват и зареждат автомати, както и да извършват различни операции като детерминизация, минимизация, обединение, конкатенация и звезда. CLI-то също така поддържа разпознаване на думи от автоматите и визуализация на автоматите чрез DOT формат.

Документация за клас генериран от следните файлове:

- AutomatonCLI.hpp
- AutomatonCLI.cpp

4.3 AutomatonSerializer Клас Препратка

Статични общодостъпни членове функции

- static std::string `serialize` (const `Automaton` &automaton)
Сериализира даден автомат в human-readable формат.
- static `Automaton` `deserialize` (std::istream &is)
Десериализира автомат от human-readable формат. Форматът трябва да съответства на този, описан в `serialize()`.
- static std::string `toDot` (const `Automaton` &automaton)
Генерира представяне на автомата в DOT формат, който може да бъде визуализиран с инструменти като Graphviz. В DOT представянето, състоянията са възли, а преходите са насочени ръбове между тях. Началните състояния са маркирани със зелени стрелки, а крайните състояния са представени с двойни кръгове.
- static void `print` (const `Automaton` &automaton, std::ostream &os=std::cout)

4.3.1 Членове Функции(методи) Документация

4.3.1.1 `serialize()`

```
std::string AutomatonSerializer::serialize (
    const Automaton & automaton) [static]
```

Сериализира даден автомат в human-readable формат.

- Форматът на сериализацията е следният:

```
<брой_символи_в_азбуката> <символи>
<общ_брой_състояния>
<състояние_1> ...
<брой_начални_състояния>
<състояние_1> ...
<брой_финални_състояния>
<състояние_1> ...
<брой_преходи>
<from> <symbol> <to>
...
```

Документация за клас генериран от следните файлове:

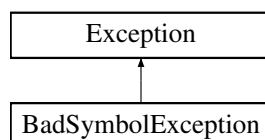
- AutomatonSerializer.hpp
- AutomatonSerializer.cpp

4.4 BadSymbolException Клас Препратка

Изключение, което се хвърля при опит за използване на символ, който не е част от азбуката на автомата. Съдържа информация за символа, който е бил използван и не е валиден.

```
#include <BadSymbolException.hpp>
```

Диаграма на наследяване за BadSymbolException:



Общодостъпни членове функции

- `BadSymbolException (char symbol)`

Общодостъпни членове функции наследен от [Exception](#)

- `Exception (const std::string &message)`
- `virtual std::string toString () const`

4.4.1 Подробно описание

Исключение, което се хвърля при опит за използване на символ, който не е част от азбуката на автомата. Съдържа информация за символа, който е бил използван и не е валиден.

Документация за клас генериран от следният файл:

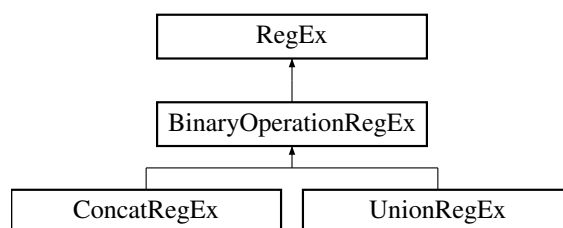
- `BadSymbolException.hpp`

4.5 BinaryOperationRegEx Клас Препратка

Класът [BinaryOperationRegEx](#) е абстрактен базов клас за представяне на бинарни операции между регулярни изрази, като обединение и конкатенация. Той съдържа два указателя към обекти от тип [RegEx](#), които представляват лявата и дясната част на операцията.

```
#include <BinaryOperationRegEx.hpp>
```

Диаграма на наследяване за `BinaryOperationRegEx`:



Общодостъпни членове функции

- `BinaryOperationRegEx (const RegEx &lhs, const RegEx &rhs)`
Конструктор, който приема два регулярни изрази. Класът копира подадените регулярни изрази.
- `BinaryOperationRegEx (RegEx *lhs, RegEx *rhs)`
Конструктор, който приема указатели към два регулярни изрази и ги присвоява на членовете на класа. Класът поема собствеността върху тези указатели и ще ги освободи при унищожаване на обекта.
- `BinaryOperationRegEx (const BinaryOperationRegEx &other)`
- `BinaryOperationRegEx & operator= (const BinaryOperationRegEx &other)`
- `BinaryOperationRegEx (BinaryOperationRegEx &&other) noexcept=default`
- `BinaryOperationRegEx & operator= (BinaryOperationRegEx &&other) noexcept=default`

Общодостъпни членове функции наследен от [RegEx](#)

- virtual bool [eval](#) (const std::string &) const =0
Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.
- virtual [RegEx](#) * clone () const =0
- virtual std::string [toString](#) () const =0
Връща низово представяне на регулярния израз.
- virtual [Automaton toAutomaton](#) () const =0
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Защитени атрибути

- UniquePtr< [RegEx](#) > lhs
- UniquePtr< [RegEx](#) > rhs

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от [RegEx](#)

- static UniquePtr< [RegEx](#) > parse (const std::string &expr)
Статична функция, която приема регулярен израз и връща указател към обект от тип [RegEx](#).

4.5.1 Подробно описание

Класът [BinaryOperationRegEx](#) е абстрактен базов клас за представяне на бинарни операции между регулярни изрази, като обединение и конкатенация. Той съдържа два указателя към обекти от тип [RegEx](#), които представляват лявата и дясната част на операцията.

Документация за клас генериран от следните файлове:

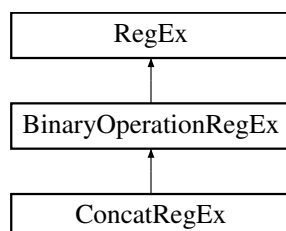
- BinaryOperationRegEx.hpp
- BinaryOperationRegEx.cpp

4.6 ConcatRegEx Клас Препратка

Класът [ConcatRegEx](#) е конкатенация на два други регулярни изрази.

```
#include <ConcatRegEx.hpp>
```

Диаграма на наследяване за ConcatRegEx:



Общодостъпни членове функции

- `ConcatRegEx (const RegEx &lhs, const RegEx &rhs)`
- `ConcatRegEx (RegEx *lhs, RegEx *rhs)`
Конструктор, който приема два регулярни изрази и създава нов обект от тип `ConcatRegEx`.
- `std::string toString () const override`
Връща низово представяне на регулярния израз.
- `Automaton toAutomaton () const override`
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.
- `bool eval (const std::string &word) const override`
Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.
- `RegEx * clone () const override`

Общодостъпни членове функции наследен от `BinaryOperationRegEx`

- `BinaryOperationRegEx (const RegEx &lhs, const RegEx &rhs)`
Конструктор, който приема два регулярни изрази. Класът копира подадените регулярни изрази.
- `BinaryOperationRegEx (RegEx *lhs, RegEx *rhs)`
Конструктор, който приема указатели към два регулярни изрази и ги присвоява на членовете на класа. Класът поема собствеността върху тези указатели и ще ги освободи при унищожаване на обекта.
- `BinaryOperationRegEx (const BinaryOperationRegEx &other)`
- `BinaryOperationRegEx & operator= (const BinaryOperationRegEx &other)`
- `BinaryOperationRegEx (BinaryOperationRegEx &&other) noexcept=default`
- `BinaryOperationRegEx & operator= (BinaryOperationRegEx &&other) noexcept=default`

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от `RegEx`

- `static UniquePtr< RegEx > parse (const std::string &expr)`
Статична функция, която приема регулярен израз и връща указател към обект от тип `RegEx`.

Защитени атрибути наследен от `BinaryOperationRegEx`

- `UniquePtr< RegEx > lhs`
- `UniquePtr< RegEx > rhs`

4.6.1 Подробно описание

Класът `ConcatRegEx` е конкатенация на два други регулярни изрази.

4.6.2 Членове Функции(методи) Документация

4.6.2.1 `clone()`

`RegEx * ConcatRegEx::clone () const [override], [virtual]`

Заменя `RegEx`.

4.6.2.2 eval()

```
bool ConcatRegEx::eval (
    const std::string & word) const    [override], [virtual]
```

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя [RegEx](#).

4.6.2.3 toAutomaton()

```
Automaton ConcatRegEx::toAutomaton () const    [override], [virtual]
```

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя [RegEx](#).

4.6.2.4 toString()

```
std::string ConcatRegEx::toString () const    [override], [virtual]
```

Връща низово представяне на регулярния израз.

Заменя [RegEx](#).

Документация за клас генериран от следните файлове:

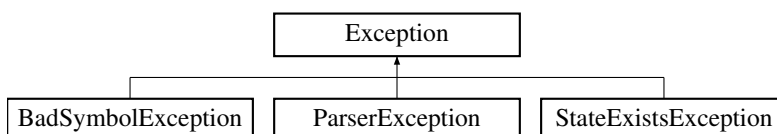
- ConcatRegEx.hpp
- ConcatRegEx.cpp

4.7 Exception Клас Препратка

Базов клас за всички изключения в проекта. Той съдържа съобщение за грешката и предоставя метод toString() за връщане на това съобщение.

```
#include <Exception.hpp>
```

Диаграма на наследяване за Exception:



Общодостъпни членове функции

- Exception (const std::string &message)
- virtual std::string toString () const

4.7.1 Подробно описание

Базов клас за всички изключения в проекта. Той съдържа съобщение за грешката и предоставя метод `toString()` за връщане на това съобщение.

Документация за клас генериран от следният файл:

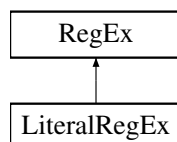
- `Exception.hpp`

4.8 LiteralRegex Клас Препратка

Класът [LiteralRegex](#) представлява регулярни изрази, които съответстват на конкретна дума.

```
#include <LiteralRegex.hpp>
```

Диаграма на наследяване за `LiteralRegex`:



Общодостъпни членове функции

- `LiteralRegex (const std::string &word)`
Конструктор, който приема дума и създава регулярен израз.
- `bool eval (const std::string &word) const override`
Връща `true`, ако дадената дума се разпознава от регулярния израз, и `false` в противен случай.
- `std::string toString () const override`
Връща низово представяне на регулярния израз.
- `Automaton toAutomaton () const override`
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.
- `Regex * clone () const override`

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от [Regex](#)

- `static UniquePtr< Regex > parse (const std::string &expr)`
Статична функция, която приема регулярен израз и връща указател към обект от тип [Regex](#).

4.8.1 Подробно описание

Класът [LiteralRegex](#) представлява регулярни изрази, които съответстват на конкретна дума.

4.8.2 Членове Функции(методи) Документация

4.8.2.1 clone()

[RegExp](#) * LiteralRegExp::clone () const [override], [virtual]

Заменя [RegExp](#).

4.8.2.2 eval()

bool LiteralRegExp::eval (
const std::string & word) const [override], [virtual]

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя [RegExp](#).

4.8.2.3 toAutomaton()

[Automaton](#) LiteralRegExp::toAutomaton () const [override], [virtual]

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя [RegExp](#).

4.8.2.4 toString()

std::string LiteralRegExp::toString () const [override], [virtual]

Връща низово представяне на регулярния израз.

Заменя [RegExp](#).

Документация за клас генериран от следните файлове:

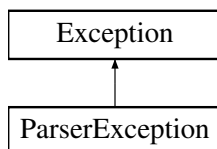
- LiteralRegExp.hpp
- LiteralRegExp.cpp

4.9 ParserException Клас Препратка

Изключение, което се хвърля при срещане на синтактична грешка по време на парсване на регулярни изрази. Съдържа текст описващ грешката, индекса където е възникнала грешката, и самия израз, който е бил парсван.

```
#include <ParserException.hpp>
```

Диаграма на наследяване за ParserException:



Общодостъпни членове функции

- `ParseException (const std::string &message, const std::string &expr, int idx)`
- `std::string toString () const` override

Общодостъпни членове функции наследен от [Exception](#)

- `Exception (const std::string &message)`

4.9.1 Подробно описание

Изключение, което се хвърля при срещане на синтактична грешка по време на парсване на регулярни изрази. Съдържа текст описващ грешката, индекса където е възникнала грешката, и самия израз, който е бил парсван.

4.9.2 Членове Функции(методи) Документация

4.9.2.1 toString()

```
std::string ParseException::toString () const [inline], [override], [virtual]
```

Заменя наследеният метод [Exception](#).

Документация за клас генериран от следният файл:

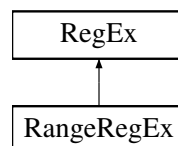
- `ParseException.hpp`

4.10 RangeRegEx Клас Препратка

Класът [RangeRegEx](#) представлява регулярни изрази, които са диапазон от символи. Например, `[a-z]` представлява всички малки букви от а до z.

```
#include <RangeRegEx.hpp>
```

Диаграма на наследяване за RangeRegEx:



Общодостъпни членове функции

- `RangeRegEx (char start, char end)`
Конструктор, който приема два символа, представляващи началото и края на диапазона, и създава нов обект от тип `RangeRegEx`.
- `bool eval (const std::string &word) const override`
Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.
- `Automaton toAutomaton () const override`
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.
- `std::string toString () const override`
Връща низово представяне на регулярния израз.
- `RegEx * clone () const override`

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от `RegEx`

- `static UniquePtr< RegEx > parse (const std::string &expr)`
Статична функция, която приема регулярен израз и връща указател към обект от тип `RegEx`.

4.10.1 Подробно описание

Класът `RangeRegEx` представлява регулярни изрази, които са диапазон от символи. Например, `[a-z]` представлява всички малки букви от а до z.

4.10.2 Членове Функции(методи) Документация

4.10.2.1 `clone()`

```
RegEx * RangeRegEx::clone () const [override], [virtual]
```

Заменя `RegEx`.

4.10.2.2 `eval()`

```
bool RangeRegEx::eval (  
    const std::string & word) const [override], [virtual]
```

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя `RegEx`.

4.10.2.3 `toAutomaton()`

```
Automaton RangeRegEx::toAutomaton () const [override], [virtual]
```

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя `RegEx`.

4.10.2.4 toString()

```
std::string RangeRegex::toString () const [override], [virtual]
```

Връща низово представяне на регулярния израз.

Заменя [Regex](#).

Документация за клас генериран от следните файлове:

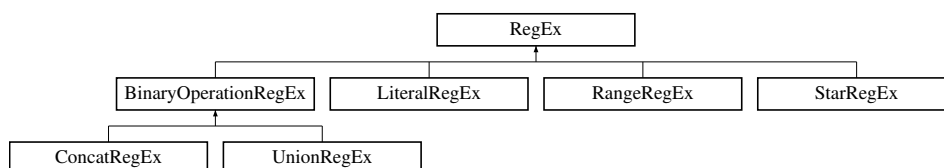
- RangeRegex.hpp
- RangeRegex.cpp

4.11 Regex Клас Препратка

Абстрактен базов клас за представяне на регулярни изрази.

```
#include <Regex.hpp>
```

Диаграма на наследяване за Regex:



Общодостъпни членове функции

- virtual bool [eval](#) (const std::string &) const =0
Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.
- virtual [Regex](#) * clone () const =0
- virtual std::string [toString](#) () const =0
Връща низово представяне на регулярния израз.
- virtual [Automaton](#) [toAutomaton](#) () const =0
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Статични общодостъпни членове функции

- static UniquePtr< [Regex](#) > parse (const std::string &expr)
Статична функция, която приема регулярен израз и връща указател към обект от тип [Regex](#).

4.11.1 Подробно описание

Абстрактен базов клас за представяне на регулярни изрази.

4.11.2 Членове Функции(методи) Документация

4.11.2.1 eval()

```
virtual bool RegEx::eval (
    const std::string & ) const    [pure virtual]
```

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя в [ConcatRegEx](#), [LiteralRegEx](#), [RangeRegEx](#), [StarRegEx](#), и [UnionRegEx](#).

4.11.2.2 toAutomaton()

```
virtual Automaton RegEx::toAutomaton () const    [pure virtual]
```

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя в [ConcatRegEx](#), [LiteralRegEx](#), [RangeRegEx](#), [StarRegEx](#), и [UnionRegEx](#).

4.11.2.3 toString()

```
virtual std::string RegEx::toString () const    [pure virtual]
```

Връща низово представяне на регулярния израз.

Заменя в [ConcatRegEx](#), [LiteralRegEx](#), [RangeRegEx](#), [StarRegEx](#), и [UnionRegEx](#).

Документация за клас генериран от следните файлове:

- [RegEx.hpp](#)
- [RegEx.cpp](#)

4.12 RegExParser Клас Препратка

Класът [RegExParser](#) използва рекурсивен парсер за обработка на различните оператори в регулярните изрази, като обединение (+), конкатенация(без оператор) и звезда (*). За празна дума се използва символът '@'.

```
#include <RegExParser.hpp>
```

Общодостъпни членове функции

- [RegExParser](#) (const std::string &expr)
Конструктор, който приема регулярен израз.
- [UniquePtr< \[RegEx\]\(#\) > parse](#) ()
Функция, която парсва регулярния израз и връща указател към обект от тип [RegEx](#), който представлява този израз.

4.12.1 Подробно описание

Класът [RegExParser](#) използва рекурсивен парсер за обработка на различните оператори в регулярните изрази, като обединение (+), конкатенация(без оператор) и звезда (*). За празна дума се използва символът '@'.

4.12.2 Членове Функции(методи) Документация

4.12.2.1 parse()

UniquePtr< [RegEx](#) > RegExParser::parse ()

Функция, която парсва регулярния израз и връща указател към обект от тип [RegEx](#), който представлява този израз.

Изключения

ParserException	Ако по време на парсването се срещне синтактична грешка, се хвърля изключение от тип ParserException , което съдържа информация за грешката и позицията в израза, където е възникнала.
-----------------	--

Документация за клас генериран от следните файлове:

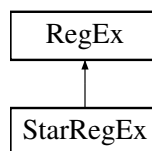
- RegExParser.hpp
- RegExParser.cpp

4.13 StarRegEx Клас Препратка

Класът [StarRegEx](#) представлява регулярни изрази, които са звезда на друг регулярен израз.

```
#include <StarRegEx.hpp>
```

Диаграма на наследяване за StarRegEx:



Общодостъпни членове функции

- `StarRegEx (const RegEx &inner)`
Конструктор, който приема друг регулярен израз и създава звезда на този израз. Класът копира подадения регулярен израз.
- `StarRegEx (RegEx *inner)`
Конструктор, който приема друг регулярен израз и създава звезда на този израз. Собствеността върху указателя се поема от класа `StarRegEx`, който ще го освободи при унищожаване на обекта.
- `StarRegEx (const StarRegEx &other)`
- `StarRegEx & operator= (const StarRegEx &other)`
- `StarRegEx (StarRegEx &&other) noexcept=default`
- `StarRegEx & operator= (StarRegEx &&other) noexcept=default`
- `bool eval (const std::string &word) const override`
Връща `true`, ако дадената дума се разпознава от регулярния израз, и `false` в противен случай.
- `Automaton toAutomaton () const override`
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.
- `std::string toString () const override`
Връща низово представяне на регулярния израз.
- `RegEx * clone () const override`

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от `RegEx`

- `static UniquePtr< RegEx > parse (const std::string &expr)`
Статична функция, която приема регулярен израз и връща указател към обект от тип `RegEx`.

4.13.1 Подробно описание

Класът `StarRegEx` представлява регулярни изрази, които са звезда на друг регулярен израз.

4.13.2 Членове Функции(методи) Документация

4.13.2.1 `clone()`

`RegEx * StarRegEx::clone () const [override], [virtual]`

Заменя `RegEx`.

4.13.2.2 `eval()`

`bool StarRegEx::eval (const std::string & word) const [override], [virtual]`

Връща `true`, ако дадената дума се разпознава от регулярния израз, и `false` в противен случай.

Заменя `RegEx`.

4.13.2.3 toAutomaton()

`Automaton` `StarRegex::toAutomaton () const` [override], [virtual]

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя [Regex](#).

4.13.2.4 toString()

`std::string` `StarRegex::toString () const` [override], [virtual]

Връща низово представяне на регулярния израз.

Заменя [Regex](#).

Документация за клас генериран от следните файлове:

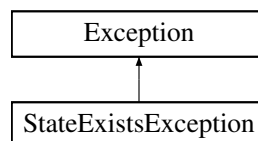
- `StarRegex.hpp`
- `StarRegex.cpp`

4.14 StateExistsException Клас Препратка

Изключение, което се хвърля при опит за създаване на състояние с идентификатор, който вече съществува в автомата, или при опит за достъп до несъществуващо състояние. Съдържа информация за идентификатора на състоянието и дали то вече съществува или не.

```
#include <StateExistsException.hpp>
```

Диаграма на наследяване за `StateExistsException`:



Общодостъпни членове функции

- `StateExistsException (size_t id, bool exists)`
- `std::string` `toString () const` override
- `bool exists () const`
- `size_t id () const`

Общодостъпни членове функции наследен от [Exception](#)

- `Exception (const std::string &message)`

4.14.1 Подробно описание

Исключение, което се хвърля при опит за създаване на състояние с идентификатор, който вече съществува в автомата, или при опит за достъп до несъществуващо състояние. Съдържа информация за идентификатора на състоянието и дали то вече съществува или не.

4.14.2 Членове Функции(методи) Документация

4.14.2.1 toString()

```
std::string StateExistsException::toString () const [inline], [override], [virtual]
```

Заменя наследеният метод [Exception](#).

Документация за клас генериран от следният файл:

- StateExistsException.hpp

4.15 Transition Структура Препратка

Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като '\0').

```
#include <Automaton.hpp>
```

Общодостъпни атрибути

- size_t from
- char symbol
- size_t to

4.15.1 Подробно описание

Преходът се представя като наредена тройка (from, symbol, to), където from и to са идентификатори на състояния, а symbol е символ от азбуката или епсилон (представен като '\0').

Документация за структура генериран от следният файл:

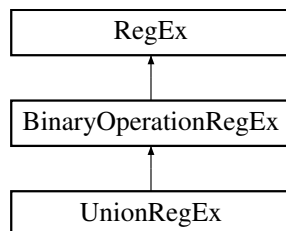
- Automaton.hpp

4.16 UnionRegEx Клас Препратка

Класът [UnionRegEx](#) представлява обединение на два други регулярни изрази.

```
#include <UnionRegEx.hpp>
```

Диаграма на наследяване за UnionRegEx:



Общодостъпни членове функции

- [UnionRegEx](#) (const [RegEx](#) &lhs, const [RegEx](#) &rhs)
Конструктор, който приема два регулярни изрази и създава нов обект от тип [UnionRegEx](#). Класът копира подадените регулярни изрази.
- [UnionRegEx](#) ([RegEx](#) *lhs, [RegEx](#) *rhs)
Конструктор, който приема указатели към два регулярни изрази и създава обект от тип [UnionRegEx](#). Класът поема собствеността върху тези указатели и ще ги освободи при унищожаване на обекта.
- `std::string toString () const override`
Връща низово представяне на регулярния израз.
- [Automaton toAutomaton \(\) const override](#)
Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.
- `bool eval (const std::string &word) const override`
Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.
- [RegEx](#) * `clone () const override`

Общодостъпни членове функции наследен от [BinaryOperationRegEx](#)

- [BinaryOperationRegEx](#) (const [RegEx](#) &lhs, const [RegEx](#) &rhs)
Конструктор, който приема два регулярни изрази. Класът копира подадените регулярни изрази.
- [BinaryOperationRegEx](#) ([RegEx](#) *lhs, [RegEx](#) *rhs)
Конструктор, който приема указатели към два регулярни изрази и ги присвоява на членовете на класа. Класът поема собствеността върху тези указатели и ще ги освободи при унищожаване на обекта.
- [BinaryOperationRegEx](#) (const [BinaryOperationRegEx](#) &other)
- [BinaryOperationRegEx](#) & operator= (const [BinaryOperationRegEx](#) &other)
- [BinaryOperationRegEx](#) ([BinaryOperationRegEx](#) &&other) noexcept=default
- [BinaryOperationRegEx](#) & operator= ([BinaryOperationRegEx](#) &&other) noexcept=default

Допълнителни наследени членове

Статични общодостъпни членове функции наследен от [RegEx](#)

- `static UniquePtr< RegEx > parse (const std::string &expr)`
Статична функция, която приема регулярен израз и връща указател към обект от тип [RegEx](#).

Защитени атрибути наследен от [BinaryOperationRegEx](#)

- UniquePtr< [RegEx](#) > lhs
- UniquePtr< [RegEx](#) > rhs

4.16.1 Подробно описание

Класът [UnionRegEx](#) представлява обединение на два други регулярни изрази.

4.16.2 Членове Функции(методи) Документация

4.16.2.1 clone()

[RegEx](#) * UnionRegEx::clone () const [override], [virtual]

Заменя [RegEx](#).

4.16.2.2 eval()

bool UnionRegEx::eval (
const std::string & word) const [override], [virtual]

Връща true, ако дадената дума се разпознава от регулярния израз, и false в противен случай.

Заменя [RegEx](#).

4.16.2.3 toAutomaton()

[Automaton](#) UnionRegEx::toAutomaton () const [override], [virtual]

Връща еквивалентен автомат, който разпознава същия език като регулярния израз. Автоматът може да бъде недетерминиран (NFA) и може да съдържа епсилон-преходи.

Заменя [RegEx](#).

4.16.2.4 toString()

std::string UnionRegEx::toString () const [override], [virtual]

Връща низово представяне на регулярния израз.

Заменя [RegEx](#).

Документация за клас генериран от следните файлове:

- UnionRegEx.hpp
- UnionRegEx.cpp

Глава 5

Файлове Документация

5.1 Automaton.hpp

```
00001 #pragma once
00002 #include <map>
00003 #include <vector>
00004 #include <string>
00005 #include <set>
00006 #include <iostream>
00007
00008 #include "../Exceptions/Exception.hpp"
00009 #include "../Exceptions/StateExistsException.hpp"
00010 #include "../Exceptions/BadSymbolException.hpp"
00011
00012 constexpr char EPSILON = '\0';
00013
00014
00015 struct Transition
00016 {
00017     size_t from;
00018     char symbol;
00019     size_t to;
00020 };
00021
00022 class Automaton
00023 {
00024 public:
00025     friend class AutomatonSerializer;
00026
00027     Automaton(const std::vector<char> alphabet);
00028
00029     void createState(size_t id, bool isStart, bool isFinal);
00030
00031     void addTransition(size_t from, char symbol, size_t to);
00032
00033     bool recognise(const std::string& word) const;
00034
00035     Automaton getReversed() const;
00036     void reverse();
00037
00038     bool isDeterministic() const;
00039     Automaton getDeterministic() const;
00040
00041     void determinise();
00042
00043     Automaton getMinimised() const;
00044
00045     void minimise();
00046
00047     friend Automaton Union(const Automaton& lhs, const Automaton& rhs);
00048
00049     friend Automaton Concat(const Automaton& lhs, const Automaton& rhs);
00050
00051     friend Automaton Star(const Automaton& automaton);
00052
00053     const std::vector<char>& getAlphabet() const
00054     {
00055         return m_alphabet;
00056     }
00057 }
```

```

00124
00125     ~Automaton() = default;
00126
00130     size_t getStatesCount() const
00131     {
00132         return m_states.size();
00133     }
00134
00135 private:
00136     bool hasState(size_t id) const
00137     {
00138         for (size_t state : m_states)
00139             if (state == id) return true;
00140         return false;
00141     }
00142     bool hasAnyTransitions(size_t state) const
00143     {
00144         for (const Transition& transition : m_transitions)
00145             if (transition.from == state) return true;
00146         return false;
00147     }
00148     bool symbolIsInAlphabet(char symbol) const
00149     {
00150         for (char s : m_alphabet)
00151         {
00152             if (s == symbol) return true;
00153         }
00154         return false;
00155     }
00156
00157
00158     void pushReachableSubsets(Automaton& dfa, std::vector<std::vector<size_t>& stateSubsets) const;
00159
00160     void addSubsetTransitions(Automaton& dfa, const std::vector<std::vector<size_t>& stateSubsets) const;
00161
00162
00163     void pushStatesWithEpsilon(std::vector<size_t>& closure, std::vector<size_t>& stack, size_t stateId) const;
00164
00165     std::vector<size_t> getEpsilonClosure(const std::vector<size_t>& states) const;
00166     std::vector<size_t> getNextStates(const std::vector<size_t>& currentStates, char symbol) const;
00167
00168     bool containsFinalState(const std::vector<size_t>& currentStates) const;
00169 private:
00170     std::vector<size_t> m_states;
00171     std::vector<size_t> m_startStates;
00172     std::vector<size_t> m_finalStates;
00173
00174     std::vector<Transition> m_transitions;
00175     std::vector<char> m_alphabet;
00176 };

```

5.2 AutomatonSerializer.hpp

```

00001 #pragma once
00002 #include <string>
00003 #include <vector>
00004 #include <iostream>
00005 #include "Automaton.hpp"
00006
00007 class AutomatonSerializer
00008 {
00009 public:
00010
00027     static std::string serialize(const Automaton& automaton);
00028
00032     static Automaton deserialize(std::istream& is);
00033
00037     static std::string toDot(const Automaton& automaton);
00038
00039     static void print(const Automaton& automaton, std::ostream& os = std::cout);
00040
00041 private:
00042     static std::string serializeStates(const std::vector<size_t>& container);
00043
00044     static std::string serializeTransitions(const Automaton& automaton);
00045
00046     static Automaton deserializeAlphabet(std::istream& is);
00047
00048     static void deserializeStates(Automaton& automaton, std::istream& is);
00049
00050     static void deserializeTransitions(Automaton& automaton, std::istream& is);
00051
00052     AutomatonSerializer() = default;
00053 };

```

5.3 BadSymbolException.hpp

```

00001 #include "Exception.hpp"
00002
00003
00007 class BadSymbolException : public Exception
00008 {
00009     char m_symbol;
00010 public:
00011     BadSymbolException(char symbol) : Exception("Symbol '" + std::string(1, symbol) + "' is not in the alphabet."),
        m_symbol(symbol) {}
00012     virtual ~BadSymbolException() = default;
00013 };

```

5.4 Exception.hpp

```

00001 #pragma once
00002 #include <string>
00003
00004
00008 class Exception
00009 {
00010     const std::string m_message;
00011 public:
00012     Exception(const std::string& message) : m_message(message) {}
00013     virtual std::string toString() const { return m_message; }
00014     virtual ~Exception() = default;
00015 };

```

5.5 ParserException.hpp

```

00001 #include "Exception.hpp"
00002
00003
00007 class ParserException : public Exception
00008 {
00009     int idx;
00010     std::string expr;
00011 public:
00012     ParserException(const std::string& message, const std::string& expr, int idx) : Exception(message), expr(expr), idx(idx)
        {}
00013     std::string toString() const override
00014     {
00015         return Exception::toString() + " at index " + std::to_string(idx) + " in expression: " + expr;
00016     }
00017 };

```

5.6 StateExistsException.hpp

```

00001 #pragma once
00002 #include "Exception.hpp"
00003
00004
00005
00009 class StateExistsException : public Exception
00010 {
00011     size_t m_id;
00012     bool m_exists;
00013 public:
00014     StateExistsException(size_t id, bool exists) : Exception(""), m_id(id), m_exists(exists) {}
00015     std::string toString() const override
00016     {
00017         return "State " + std::to_string(m_id) + (m_exists ? " already exists." : " does not exist.");
00018     }
00019
00020     bool exists() const { return m_exists; }
00021     size_t id() const { return m_id; }
00022     virtual ~StateExistsException() = default;
00023 };

```

5.7 BinaryOperationRegEx.hpp

```

00001 #pragma once
00002 #include "RegEx.hpp"
00003 #include "../Utils/UniquePtr.hpp"
00004
00008 class BinaryOperationRegEx : public RegEx
00009 {
00010 protected:
00011     UniquePtr<RegEx> lhs;
00012     UniquePtr<RegEx> rhs;
00013 public:
00018     BinaryOperationRegEx(const RegEx& lhs, const RegEx& rhs);
00022     BinaryOperationRegEx(RegEx* lhs, RegEx* rhs);
00023     BinaryOperationRegEx(const BinaryOperationRegEx& other);
00024     BinaryOperationRegEx& operator=(const BinaryOperationRegEx& other);
00025
00026     BinaryOperationRegEx(BinaryOperationRegEx&& other) noexcept = default;
00027     BinaryOperationRegEx& operator=(BinaryOperationRegEx&& other) noexcept = default;
00028     virtual BinaryOperationRegEx() = default;
00029 };

```

5.8 ConcatRegEx.hpp

```

00001 #pragma once
00002 #include "BinaryOperationRegEx.hpp"
00003
00004
00008 class ConcatRegEx : public BinaryOperationRegEx
00009 {
00010 public:
00011
00012     ConcatRegEx(const RegEx& lhs, const RegEx& rhs) : BinaryOperationRegEx(lhs, rhs) {}
00016     ConcatRegEx(RegEx* lhs, RegEx* rhs) : BinaryOperationRegEx(lhs, rhs) {}
00017
00018     std::string toString() const override;
00019     Automaton toAutomaton() const override;
00020     bool eval(const std::string& word) const override;
00021     RegEx* clone() const override;
00022 };

```

5.9 LiteralRegEx.hpp

```

00001 #pragma once
00002 #include "RegEx.hpp"
00003
00004
00008 class LiteralRegEx : public RegEx
00009 {
00010 public:
00011
00015     LiteralRegEx(const std::string& word);
00016
00017     bool eval(const std::string& word) const override;
00018
00019     std::string toString() const override;
00020
00021     Automaton toAutomaton() const override;
00022
00023     RegEx* clone() const override;
00024 private:
00025     std::string m_word;
00026 };

```

5.10 RangeRegEx.hpp

```

00001 #include "RegEx.hpp"
00002
00006 class RangeRegEx : public RegEx
00007 {
00008 public:
00012     RangeRegEx(char start, char end);
00013     bool eval(const std::string& word) const override;
00014     Automaton toAutomaton() const override;

```



```

00015     std::string toString() const override;
00016     RegEx* clone() const override;
00017 private:
00018     char m_start;
00019     char m_end;
00020 };

```

5.11 RegEx.hpp

```

00001 #pragma once
00002 #include <string>
00003 #include "../Automaton/Automaton.hpp"
00004 #include "../Utils/UniquePtr.hpp"
00005
00009 class RegEx
00010 {
00011 public:
00012
00016     virtual bool eval(const std::string&) const = 0;
00017     virtual RegEx* clone() const = 0;
00018
00022     virtual std::string toString() const = 0;
00023
00027     virtual Automaton toAutomaton() const = 0;
00028
00029     virtual ~RegEx() = default;
00030
00034     static UniquePtr<RegEx> parse(const std::string& expr);
00035 };

```

5.12 RegExParser.hpp

```

00001 #pragma once
00002 #include "RegEx.hpp"
00003 #include "StarRegEx.hpp"
00004 #include "UnionRegEx.hpp"
00005 #include "ConcatRegEx.hpp"
00006 #include "LiteralRegEx.hpp"
00007 #include "RangeRegEx.hpp"
00008
00009
00013 class RegExParser
00014 {
00015 public:
00019     RegExParser(const std::string& expr);
00020
00025     UniquePtr<RegEx> parse();
00026 private:
00027     UniquePtr<RegEx> parseUnion();
00028     UniquePtr<RegEx> parseConcat();
00029     UniquePtr<RegEx> parseStar();
00030     UniquePtr<RegEx> parseParentheses();
00031     UniquePtr<RegEx> parseBrackets();
00032     UniquePtr<RegEx> parseLiteral();
00033     std::string m_expr;
00034     size_t m_idx;
00035     char peek() const;
00036     char get();
00037 };

```

5.13 StarRegEx.hpp

```

00001 #pragma once
00002 #include "RegEx.hpp"
00003 #include "../Utils/UniquePtr.hpp"
00004
00005
00009 class StarRegEx : public RegEx
00010 {
00011 public:
00015     StarRegEx(const RegEx& inner);
00019     StarRegEx(RegEx* inner);
00020     StarRegEx(const StarRegEx& other);
00021     StarRegEx& operator=(const StarRegEx& other);

```

```

00022
00023     StarRegex(StarRegex&& other) noexcept = default;
00024     StarRegex& operator=(StarRegex&& other) noexcept = default;
00025     virtual ~StarRegex() = default;
00026
00027     bool eval(const std::string& word) const override;
00028     Automaton toAutomaton() const override;
00029     std::string toString() const override;
00030     Regex* clone() const override;
00031 private:
00032     UniquePtr<Regex> inner;
00033 };

```

5.14 UnionRegex.hpp

```

00001 #pragma once
00002 #include "BinaryOperationRegex.hpp"
00003
00004
00005 class UnionRegex : public BinaryOperationRegex
00006 {
00007 public:
00008     UnionRegex(const Regex& lhs, const Regex& rhs);
00009     UnionRegex(Regex* lhs, Regex* rhs);
00010
00011     std::string toString() const override;
00012     Automaton toAutomaton() const override;
00013     bool eval(const std::string& word) const override;
00014     Regex* clone() const override;
00015 };

```

5.15 AutomatonCLI.hpp

```

00001 #pragma once
00002 #include <iostream>
00003 #include <fstream>
00004 #include <string>
00005
00006 #include "../Automaton/Automaton.hpp"
00007 #include "../Regex/UnionRegex.hpp"
00008 #include "../Regex/ConcatRegex.hpp"
00009 #include "../Regex/StarRegex.hpp"
00010 #include "../Regex/LiteralRegex.hpp"
00011 #include "../Regex/RegexParser.hpp"
00012 #include "../Automaton/AutomatonSerializer.hpp"
00013
00014
00015 class AutomatonCLI
00016 {
00017 public:
00018     struct IdAutomaton
00019     {
00020         size_t id;
00021         Automaton automaton;
00022     };
00023     std::vector<IdAutomaton> m_automatons;
00024     Automaton& getAutomatonById(size_t id);
00025     void removeAutomatonById(size_t id);
00026     void addAutomaton(const Automaton& automaton);
00027     bool m_isRunning = true;
00028     size_t idCounter = 0;
00029     size_t getNextId();
00030     void reset();
00031 public:
00032     void run();
00033     void helpCmd();
00034     void exitCmd();
00035     void openCmd();
00036     void saveCmd();
00037     void saveAllCmd();
00038     void listCmd();
00039     void printCmd();
00040     void removeCmd();
00041     void resetCmd();
00042     void deterministicCmd();
00043     void determiniseCmd();
00044     void reverseCmd();
00045     void minimiseCmd();
00046     void recogniseCmd();

```

```
00050     void unionCmd();
00051     void concatCmd();
00052     void starCmd();
00053     void regCmd();
00054     void drawCmd();
00055 };
```


Азбучен указател

- addTransition
 - Automaton, [9](#)
- Automaton, [7](#)
 - addTransition, [9](#)
 - Automaton, [8](#)
 - Concat, [11](#)
 - createNewState, [9](#)
 - getAlphabet, [9](#)
 - getReversed, [9](#)
 - getStatesCount, [10](#)
 - isDeterministic, [10](#)
 - recognise, [10](#)
 - Star, [11](#)
 - Union, [11](#)
- Automaton.hpp, [31](#)
- AutomatonCLI, [12](#)
- AutomatonCLI.hpp, [36](#)
- AutomatonSerializer, [13](#)
 - serialize, [13](#)
- AutomatonSerializer.hpp, [32](#)
- BadSymbolException, [13](#)
- BadSymbolException.hpp, [33](#)
- BinaryOperationRegEx, [14](#)
- BinaryOperationRegEx.hpp, [34](#)
- clone
 - ConcatRegEx, [16](#)
 - LiteralRegEx, [19](#)
 - RangeRegEx, [21](#)
 - StarRegEx, [25](#)
 - UnionRegEx, [29](#)
- Concat
 - Automaton, [11](#)
- ConcatRegEx, [15](#)
 - clone, [16](#)
 - eval, [16](#)
 - toAutomaton, [17](#)
 - toString, [17](#)
- ConcatRegEx.hpp, [34](#)
- createNewState
 - Automaton, [9](#)
- eval
 - ConcatRegEx, [16](#)
 - LiteralRegEx, [19](#)
 - RangeRegEx, [21](#)
 - RegEx, [23](#)
 - StarRegEx, [25](#)
 - UnionRegEx, [29](#)
- Exception, [17](#)
- Exception.hpp, [33](#)
- getAlphabet
 - Automaton, [9](#)
- getReversed
 - Automaton, [9](#)
- getStatesCount
 - Automaton, [10](#)
- isDeterministic
 - Automaton, [10](#)
- LiteralRegEx, [18](#)
 - clone, [19](#)
 - eval, [19](#)
 - toAutomaton, [19](#)
 - toString, [19](#)
- LiteralRegEx.hpp, [34](#)
- parse
 - RegExParser, [24](#)
- ParserException, [19](#)
 - toString, [20](#)
- ParserException.hpp, [33](#)
- RangeRegEx, [20](#)
 - clone, [21](#)
 - eval, [21](#)
 - toAutomaton, [21](#)
 - toString, [21](#)
- RangeRegEx.hpp, [34](#)
- recognise
 - Automaton, [10](#)
- RegEx, [22](#)
 - eval, [23](#)
 - toAutomaton, [23](#)
 - toString, [23](#)
- RegEx.hpp, [35](#)
- RegExParser, [23](#)
 - parse, [24](#)
- RegExParser.hpp, [35](#)
- serialize
 - AutomatonSerializer, [13](#)
- Star
 - Automaton, [11](#)
- StarRegEx, [24](#)
 - clone, [25](#)
 - eval, [25](#)

- [toAutomaton](#), [25](#)
 - [toString](#), [26](#)
- [StarRegEx.hpp](#), [35](#)
- [StateExistsException](#), [26](#)
 - [toString](#), [27](#)
- [StateExistsException.hpp](#), [33](#)
- [toAutomaton](#)
 - [ConcatRegEx](#), [17](#)
 - [LiteralRegEx](#), [19](#)
 - [RangeRegEx](#), [21](#)
 - [RegEx](#), [23](#)
 - [StarRegEx](#), [25](#)
 - [UnionRegEx](#), [29](#)
- [toString](#)
 - [ConcatRegEx](#), [17](#)
 - [LiteralRegEx](#), [19](#)
 - [ParserException](#), [20](#)
 - [RangeRegEx](#), [21](#)
 - [RegEx](#), [23](#)
 - [StarRegEx](#), [26](#)
 - [StateExistsException](#), [27](#)
 - [UnionRegEx](#), [29](#)
- [Transition](#), [27](#)
- [Union](#)
 - [Automaton](#), [11](#)
- [UnionRegEx](#), [28](#)
 - [clone](#), [29](#)
 - [eval](#), [29](#)
 - [toAutomaton](#), [29](#)
 - [toString](#), [29](#)
- [UnionRegEx.hpp](#), [36](#)